

Anthropic Harness Design

for Long-Running Application Development

深度研究与工程分析报告

核心问题

为什么更强的 Coding Model 仍然需要 Harness？如何用 Planner、Generator、Evaluator、Contract、环境证据和上下文策略，把“能写代码”升级为“能长时间自主构建完整应用”？

基于 Anthropic 2026-03-24 官方工程文章，并结合其 2025 长运行 Harness、Agent Evals 与 2026 Dynamic Workflows 后续材料交叉分析

研究日期：2026-08-13

执行摘要：先给结论

一句话结论

这篇文章真正讨论的不是“三个 Agent 比一个 Agent 更强”，而是：把模型最容易犯错的环节改造成结构化、可验证、可反复迭代的闭环。Harness 的价值来自“控制流 + 状态 + 验收契约 + 独立验证 + 环境证据”，而不是 Agent 数量本身。

Anthropic 的实验路径经历了三次明显演进：2025 年先用 Initializer + Coding Agent + Fresh Context + 外部状态解决跨上下文长任务；2026 年 3 月进一步用 Planner + Generator + Evaluator 解决 scope、self-evaluation 和最后一公里质量；当 Opus 4.6 的长程能力增强后，又通过消融实验删除了 sprint 结构，把 Harness 重新简化。这个过程说明：Harness 不是固定架构，而是针对当前模型能力缺口的动态补偿层。[A1][A2][A5]

核心判断	本文分析
Harness 的本质	模型外的软件控制系统：负责上下文、工具、状态、控制流、验证、恢复和停止条件。
最关键的结构变化	Generator 与 Evaluator 分离；“做工作”和“判断是否完成”不再由同一个上下文中的同一个 Agent 完成。
最重要的工程机制	Sprint/Work Contract：先冻结 Definition of Done，再允许生成器编码，避免完成标准随实现漂移。
最重要的验证原则	Outcome > Self-report。真正的结果来自运行环境、浏览器、API、数据库和测试，而不是 Agent 说“完成了”。
最容易误读之处	更强模型不会让 Harness 彻底消失，而会让某些旧 scaffolding 失去必要性，同时把可攻克任务边界推向更复杂区域。
与 Ralph/Graph 的关系	Ralph 强调持续迭代；本文 Harness 强调角色分离与评价闭环；进一步发展自然走向 Graph of Loops。

最值得带走的 8 条原则

1. 先定义“完成”再开始编码：Contract before Code。
2. Generator 不应拥有最终验收权；Evaluator 必须尽可能独立。
3. Evaluator 不能只是读代码，要能操作真实环境并收集行为证据。
4. Context 是工作内存，文件/Git/数据库/测试结果才是耐久状态。

- 5. 失败必须分类；代码错、计划错、环境错、验收错不能全部回到同一个 Retry。
- 6. 每个 Harness 组件都隐含一个“模型做不到什么”的假设，必须通过消融实验持续复核。
- 7. 模型升级后要删 scaffolding，而不是只往 Harness 上继续堆组件。
- 8. Harness 的上限很大程度由 Verifier/Evaluator 的质量决定，而不仅由 Generator 的能力决定。

1. 文章背景与研究问题

Anthropic 于 2026 年 3 月 24 日发布《Harness design for long-running application development》，作者 Prithvi Rajasekaran 来自 Anthropic Labs。文章连接了两个看似不同的问题：让 Claude 产出更高质量的前端设计，以及让 Claude 在无人干预下长时间构建完整应用。作者发现，两类任务都在“模型很难可靠评价自己的产出”这一点上发生汇合，于是引入 Generator / Evaluator 分离结构。
[A1]

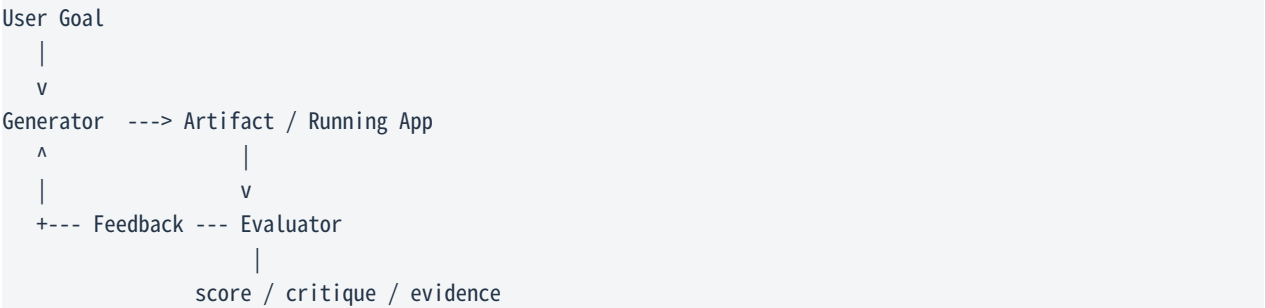
文章不是论文式 benchmark，而是以工程实验、trace 观察、prompt 调优和 Harness 消融为主要方法。因此它的价值主要在“工程方法论”和“失败模式揭示”，而不是提供可直接外推到所有 Coding Agent 的统计结论。

1.1 两个原始问题为何能够统一

问题域	表面难点	底层共性
Frontend Design	质量带主观性；“好看”难二元验收	同一生成器容易给自己的结果过高评价，需要外部标准和独立 judge。
Long-running Coding	长时间执行、scope 扩张、功能遗漏、端到端 bug	模型会把“看起来完成”误当“行为完成”，需要独立 QA 与环境证据。

1.2 作者借鉴 GAN ，但不是在做 GAN

作者借鉴的是 GAN 的角色分离直觉：一方生成，一方评价，并让评价结果反向推动生成改进。这里没有梯度、损失函数或神经网络对抗训练；它是一个软件编排层面的“生成 - 评价 - 反馈”闭环。



来源：[A1] Anthropic, Harness design for long-running application development, 2026-03-24

2. 为什么“直接让一个 Agent 一直做”会失败

2.1 Failure Mode A：长任务中的 coherence degradation

文章指出，复杂任务执行时间变长后，Agent 可能逐渐偏离原始目标。上下文不断积累实现细节、日志、错误输出、旧计划和临时决策，信息并非越多越好；有效约束在越来越多的噪声中变得更难保持稳定。Anthropic 将其中一种现象称为 context anxiety：某些模型在接近其“认为”的上下文极限时，会提前收尾、缩短探索、过早宣布完成。[A1]

Compaction 与 Context Reset 的差异

机制	Compaction	Context Reset
做法	压缩早期历史，保留同一会话继续执行	结束旧上下文，用结构化 handoff 启动新 Agent/新上下文
优势	连续性强、编排简单	真正清空历史负担，减少目标漂移与 context anxiety
风险	压缩损失；旧轨迹仍可能影响行为	handoff 若缺字段，新 Agent 可能失去关键状态
适用判断	模型能稳定长程工作、旧上下文仍有价值	严重污染、目标漂移或模型对长上下文表现不稳定

重要演进

早期 Sonnet 4.5 Harness 需要 Context Reset；文章中 Opus 4.5 已能在单个持续会话里配合自动 compaction 完成成长 build，因此作者移除了 reset。这直接说明 Context Reset 不是教条，而是能力相关的策略。[A1]

2.2 Failure Mode B：Self-Evaluation Bias

第二个问题更关键：Agent 对自己刚生成的结果往往过于宽容。它可能发现问题，却把问题解释成“不重要”；或者只做浅层 happy-path 测试，最终仍然批准工作。对于设计这类主观任务尤其明显，但文章显示在可验证的软件任务上同样存在。[A1]

同一 Agent 自评：

Generate -> 形成实现假设 -> Review 自己的实现 -> 仍沿用相同假设 -> 容易 False PASS

独立 Evaluator：

Generate -> Artifact -> 独立上下文 + Rubric + 环境操作 -> 更容易发现生成路径盲点

工程含义

“请仔细检查你的工作” 仍属于 Prompt Engineering；“你不能批准自己，由另一个 Evaluator 基于冻结的 Contract 和运行时证据验收” 才是 Harness Engineering。

3. Frontend 实验：如何把主观质量变成可操作反馈

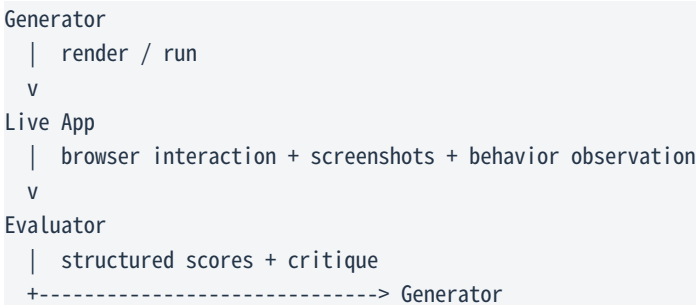
作者先在前端设计上验证 Generator-Evaluator 架构。难点在于设计质量不能简单用 unit test 表达，于是把抽象的审美判断拆成可评分的 criteria，并通过 few-shot 示例校准 Evaluator，使评分标准更稳定。
[A1]

Criterion	文章中的含义	为什么重要
Design Quality	整体视觉是否形成统一、清晰的身份和氛围	避免组件拼接感；评价整体一致性。
Originality	是否存在有意的定制决策，而非模板/默认组件/典型 AI 风格	推动模型跳出安全但平庸的默认解。
Craft	字体层级、间距、配色、对比等基本功	保证技术执行不破底线。
Functionality	用户是否理解界面并完成主要任务	把“好看”与“能用”分开评价。

作者刻意提高 Design Quality 和 Originality 的权重，因为 Claude 在 Craft 和 Functionality 上本就相对更稳，而默认设计容易保守、模板化。这个实验说明 Rubric 不只是“测量仪器”，它本身也会塑造 Generator 的搜索方向。[A1]

3.1 Evaluator 为什么必须能操作环境

Evaluator 获得 Playwright MCP，能够打开真实页面、导航、点击、截图并观察实际交互后再评分。这样它不只是“LLM Critic”，而是“LLM + 工具 + 环境 + Rubric”的执行型评估器。文章中的设计循环通常进行 5-15 轮，完整运行可持续数小时。[A1]



3.2 Refine 与 Pivot：这不是普通 Retry

作者还要求 Generator 根据评分趋势决定“继续细化当前方向”还是“彻底换方向”。这使循环从机械重试变成搜索过程：当局部改进空间变小，系统允许跳出当前解附近，尝试新的设计区域。工程上可理解为 local optimization + strategic restart。

值得注意的负面现象

Rubric 的措辞也会造成视觉收敛：当评价语言过强地暗示某类风格，Generator 会为了得分逐渐靠向同一类解。这提示了 Goodhart 风险：评价指标一旦成为优化目标，就可能塑造而非仅仅测量质量。

4. 扩展到 Full-Stack：Planner + Generator + Evaluator



4.1 Planner：解决 under-specification，而不是替 Generator 写实现

旧 Harness 要求用户一开始提供详细 spec；新架构加入 Planner，把 1-4 句需求扩展成完整产品规格。作者特别要求 Planner 聚焦产品范围和高层技术设计，不要过早规定粒度过细的实现细节，因为错误的早期技术假设会被下游放大。[A1]

Planner 应明确	Planner 应避免过度规定
产品目标、用户故事、功能范围、主要模块、验收方向、高层架构边界	具体类名、每个函数签名、过细数据库字段、强绑定某个局部实现、尚未验证的技术细节

Constraint Altitude

最优规格层级通常是：明确 WHAT / WHY / acceptance boundary，同时给 Generator 保留 HOW 的实现自由。约束太少会 under-scope；约束太细会把 Planner 的早期错误固化成系统性约束。

4.2 Generator：从“无限写代码”变成受契约约束的 Builder

第一版完整 Harness 在 Opus 4.5 上仍采用 sprint/one-feature-at-a-time 的分解策略。Generator 每个 sprint 只处理一块工作，自评后再交给 QA；Git 用于版本控制。这继承了 2025 长运行 Harness 的核心经验：复杂任务要减少单次工作单元，并让每次迭代留下可恢复状态。[A1][A2]

4.3 Evaluator：独立 QA + Hard Threshold

Evaluator 通过 Playwright 以真实用户方式操作应用，并覆盖 UI、API、数据库状态。它不是给一个总平均分，而是针对 Product Depth、Functionality、Visual Design、Code Quality 等维度设硬阈值；任一关键维度不过线，sprint 即失败。[A1]

```
PASS = (depth >= T1) AND
        (functionality >= T2) AND
        (visual_design >= T3) AND
        (code_quality >= T4)
```

这种 AND Gate 比平均分更适合软件验收，因为“界面 10 分 + 代码 10 分 + 核心功能 2 分”的产品不能因为平均值尚可就进入完成态。

5. Sprint Contract：全文最值得复制的机制之一

在每个 sprint 开始编码前，Generator 先提出“本轮具体要交付什么，以及如何验证”；Evaluator 审查这个 proposal，双方迭代直到对 Definition of Done 达成一致。之后 Generator 才能编码。[A1]

```

Product Spec (high level)
  |
  v
Generator proposes Work Contract
  |
  v
Evaluator checks:
- scope correct?
- behaviors testable?
- verification strong enough?
  |
  reject / accept
  |
  v
Freeze Contract -> Coding starts

```

5.1 Contract 解决的根问题：Moving Goalposts

没有 Contract 时，Generator 可以在实现后反过来缩小“完成”的定义。例如“文件上传”被实现成 POST /upload 返回 200，然后它就把功能标记完成；但真实产品要求可能还包括前端选择、进度、错误提示、持久化、刷新后可见和下载。Contract 把完成标准冻结在编码之前，避免实现难度反向影响验收标准。

5.2 可迁移的 Work Contract Schema

```

{
  "goal": "Implement user login",
  "scope": ["UI", "API", "session persistence"],
  "acceptance": [
    "valid credentials login successfully",
    "invalid credentials show actionable error",
    "session survives refresh",
    "logout invalidates session"
  ],
  "verification": ["unit", "api", "browser_e2e"],
  "forbidden_shortcuts": ["stub-only", "mock-only completion"],
  "evidence_required": ["test results", "runtime observations"]
}

```

工程化建议

把 Contract 当作 Agent 间的接口协议，而不是普通 prompt 文本。Contract 应可版本化、可 diff、可追踪到最终 evidence，并且 Generator 无权在实现后单方面修改验收条件。

6. Agent 通信：为什么 Structured Artifact 优于共享大对话

文章中 Generator 与 Evaluator 通过文件交流：一方写 artifact，另一方读取并响应。这让上下文交接从“聊天语义”升级为“持久化协议”。与 2025 Harness 使用 progress 文件、feature list 和 Git 的思路一致：对长任务而言，Conversation 更适合作为工作内存，Artifacts 才适合作为 durable state。[A1][A2]

Conversation Memory	Structured Artifact
临时、噪声多、与当前上下文强绑定	持久、结构化、可单独读取
难 diff、难重放	可版本化、可 diff、可 replay
容易把无关历史一起带入新 Agent	只交接必要状态，降低 context pollution
“记住上一轮” 依赖模型上下文	“恢复上一轮” 依赖显式状态

关键抽象

Artifact 可以视为 Agent-to-Agent API。成熟多 Agent 系统不一定需要“所有 Agent 共享一个聊天室”，更合理的模式往往是：Agent A -> typed artifact -> Agent B。

7. Evaluator 并不天然可靠：Verifier 也必须被工程化

文章非常坦诚：Claude 默认并不是优秀 QA。早期 Evaluator 会发现真实 bug 后又自我说服“问题不大”，也会只做表面测试而漏掉深层交互。作者通过读取 QA trace，找到“Evaluator 判断与人工判断不一致”的案例，再持续更新 QA prompt，经过多轮才达到可接受水平。[A1]

7.1 把 Evaluator 当成一个分类器

	实际通过	实际失败
Evaluator 判通过	True Pass	False Pass - 最危险
Evaluator 判失败	False Reject	True Reject

对长运行 Coding Agent，False Pass 的危害远大于一般误报：一个错误功能被标 DONE 后，会成为后续功能的依赖，导致错误复利。最终系统可能得到大量“状态上已完成、行为上没完成”的功能。

7.2 文章中的三个典型 QA 发现

类型	现象	为什么代码阅读不够
交互 wiring bug	矩形填充函数存在，但 mouse-up 路径未正确触发，用户只能填起止点	“实现了函数”不等于真实用户路径调用到了函数。
状态条件 bug	删除实体的 handler 对 selection 状态要求错误，点击实体后条件不成立	局部代码合理，但状态组合不满足真实交互。
路由顺序 bug	FastAPI 的具体 reorder 路由被参数路由抢先匹配，运行时返回 422	静态看 endpoint 都存在，只有真实请求才能暴露路由匹配问题。

来源：[A1] 文章示例问题，均为本文转述而非原文长引用

7.3 Evidence Ladder：推荐的验证层级

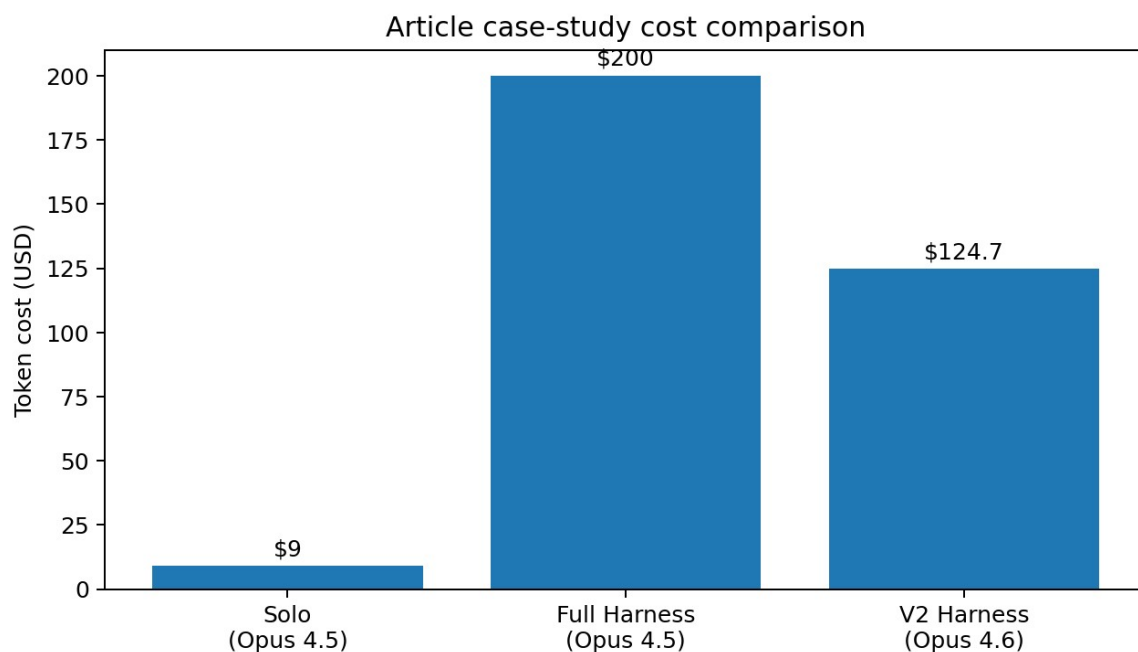
- 9. 静态检查：lint、type check、schema、build。
- 10. 确定性单元测试：核心算法和局部接口。
- 11. Integration/API：真实依赖关系、路由、持久化。
- 12. Browser/E2E：从用户视角操作完整路径。
- 13. LLM Evaluator：评价难以二元化的 product depth、UX、代码质量。
- 14. Human Gate：高风险、主观质量或 Evaluator 置信度不足时升级。

Outcome > Transcript

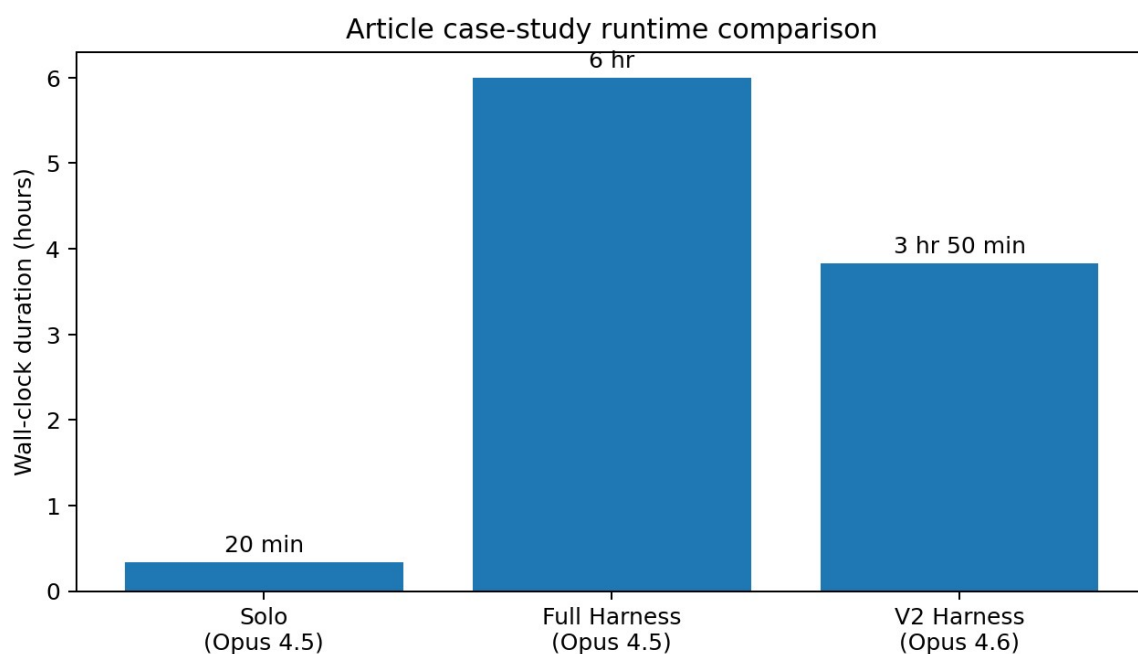
Anthropic 的 Agent Evals 文章明确区分 transcript 与 outcome：Agent 说“已经成功”只是轨迹中的语言，真正 outcome 是环境最终是否存在正确状态。长运行 Harness 应以 outcome/evidence 为判定依据。[A4]

8. 实验结果：质量提升的代价非常高

Retro Game Maker 实验中，Solo 使用 Opus 4.5 运行约 20 分钟、成本约 9 美元；完整 Harness 运行约 6 小时、成本约 200 美元，成本超过 20 倍。作者观察到 Harness 版本的核心玩法实际可运行，而 Solo 版本虽表面像完整产品，但核心游戏交互无法真正工作。[A1]



来源：数据来自 [A1]；为单次文章案例，不代表通用成本倍率



来源：数据来自 [A1]；wall-clock 时间包含 Agent 与环境交互

8.1 不能把案例实验误读成 Benchmark

- 文章没有给出大规模任务集、重复 trial、置信区间或统计显著性，因此无法证明“任何任务都应使用三 Agent Harness”。
- 它更适合支持一个工程结论：当任务位于单 Agent 能力边界之外时，规划、独立评估和运行时验证可以带来非常明显的质量提升，但代价是大量 token、延迟和编排复杂度。

- 选择 Harness 的正确目标不是“最大智能”，而是最大化 Quality Gain / Cost / Latency / Operational Complexity 的综合收益。

9. 全文最成熟的方法论：Ablation，而不是不断堆 Agent

第一版 Harness 虽有效，但庞大、慢且昂贵。作者随后不是继续增加 Supervisor、Memory、Debate，而是开始逐个删除组件并观察最终质量变化。文章提出一个很有力量的观点：Harness 每个组件都编码了一个“模型自己做不到什么”的假设；模型进步后，这些假设会过期。[A1]

```
Full Harness
|
remove one component
|
run realistic task + inspect traces
|
quality materially worse?
|      \
yes      no
|        |
keep it  delete it
```

9.1 Opus 4.6 到来后：删除 Sprint

Anthropic 发布 Opus 4.6 后，作者认为它在规划、长程 agentic execution、大代码库导航、code review、debugging 和长上下文方面的提升，正好覆盖了旧 Harness 曾经补偿的能力缺口。[A1][A5]

于是 V2 先删除 sprint construct，让 Builder 在一个长 build 中自行组织工作；Planner 与 Evaluator 被保留。Evaluator 也从“每 sprint 检查”改为“build 后单次 QA”，因为更强模型已经能独立处理很多过去需要外部检查的任务。[A1]

最关键的动态边界

Evaluator 不是固定必选项。任务如果落在当前模型可以可靠 solo 的能力区域，Evaluator 只是成本；任务越靠近或超出模型可靠边界，独立评价带来的收益越明显。

9.2 Harness 的真正函数

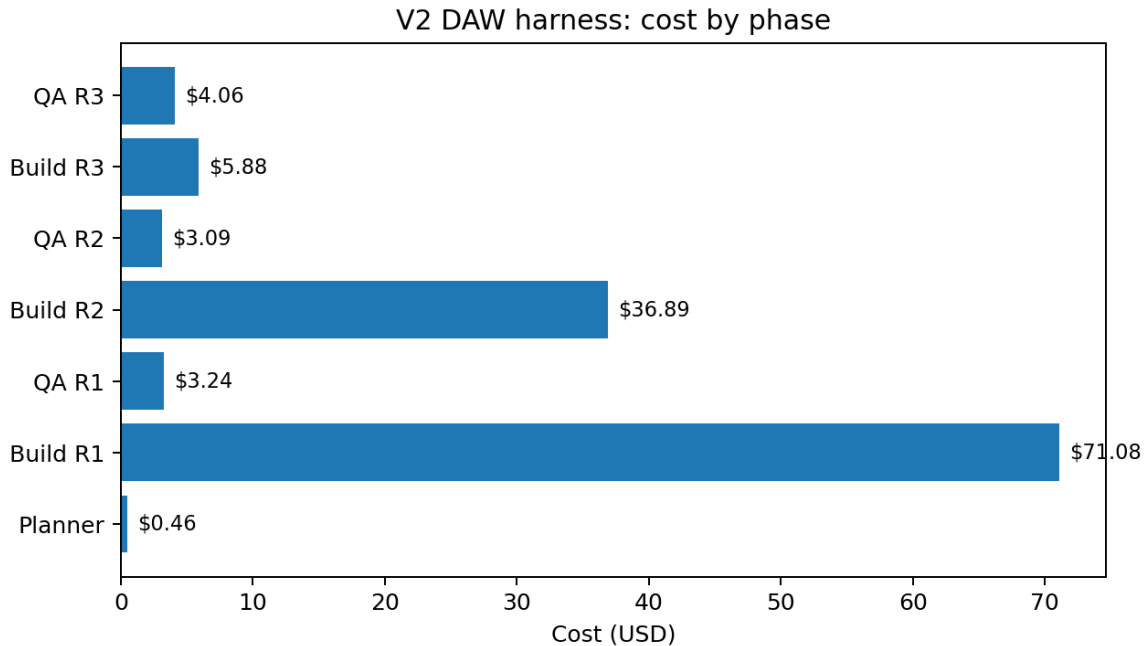
Harness = f(Model Capability, Task Complexity, Risk, Environment, Budget)

不是：

Harness = Planner + Generator + Evaluator （永远固定）

10. V2 DAW 实验：模型变强后，Harness 如何变瘦

V2 使用 Opus 4.6 构建浏览器 DAW。总耗时约 3 小时 50 分，总 token 成本 124.70 美元，其中绝大部分成本来自 Builder；第一轮 Build 连续运行 2 小时 7 分，不再依赖 Opus 4.5 时的 sprint decomposition。[A1]



来源：数据来自 [A1] 表格，四舍五入显示

10.1 为什么 QA 仍然有价值

即使 Builder 已经能长期连贯工作，QA 仍发现多个“Surface Complete / Behavior Incomplete”问题：例如录音按钮只是切换状态却没有真实麦克风采集、clip 移动/缩放/切分缺失、效果器只有数字 slider 而没有规格要求的图形化交互。也就是说，模型仍会用“有 UI 元素”替代“核心交互完整”。[A1]

Surface Completion	Behavioral Completion
按钮存在	按钮触发真实端到端能力
API endpoint 存在	正确参数、路由、持久化、错误路径都可工作
Effect 有 slider	满足 spec 中要求的可视化和交互深度
页面能加载	关键用户任务可以完整完成

核心认识

“看起来像产品”与“功能上是产品”之间存在巨大鸿沟。长运行 Agent 的最后一公里往往不是生成更多代码，

而是构建足够强的证据链证明核心行为真的存在。

11. 与 2025 Long-Running Harness、Ralph Loop 的关系

11.1 2025 Anthropic Harness 的核心

2025 年方案解决的是跨上下文持续进展：Initializer 首次设置 init.sh、progress 文件、Git 和 feature list；后续 Coding Agent 每个 session 只做增量工作，并通过显式 artifacts 给下一 session 恢复状态。feature list 一开始全部标记 failing，Agent 只能把 passes 状态改为 true，不能删除验收项。[A2]

```
Initializer
-> init.sh
-> feature_list.json
-> progress.txt
-> git baseline

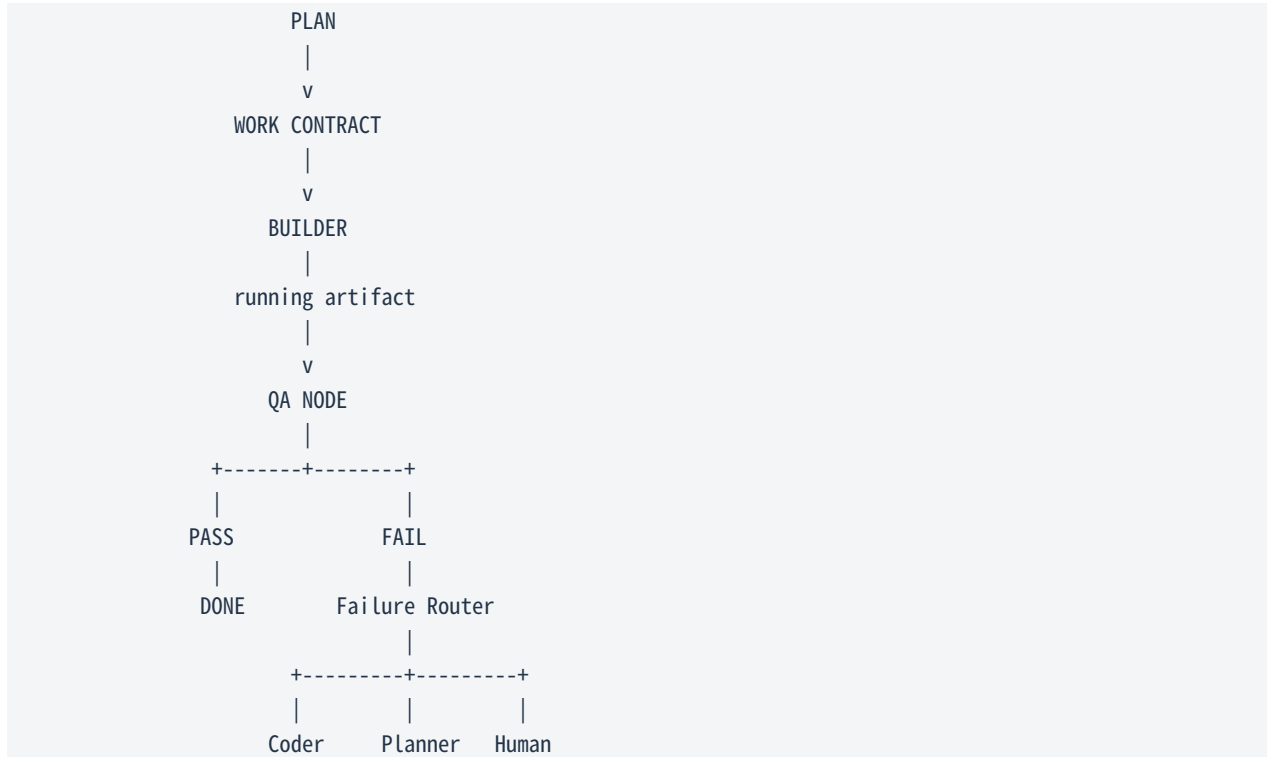
Fresh Coding Session N
-> read state
-> run smoke test
-> pick one incomplete feature
-> implement + E2E verify
-> commit + update artifacts
-> handoff to Session N+1
```

11.2 Ralph Loop 与本文 Harness 的互补

维度	Ralph Loop	Anthropic 2026 Harness
主要问题	如何让 Agent 持续迭代，不靠人不断催促	如何让长构建在 scope、评价和验收上保持可靠
关键循环	Agent -> action -> verify -> next iteration	Planner -> Generator -> Evaluator -> feedback -> Generator
状态	Git / files / progress / tests	Spec / Contract / files / Git / QA evidence
最大风险	Verifier 弱导致错误不断复利	Evaluator 宽松、scope 错误、成本过高
本质	Persistent Iteration	Role Separation + Evidence-based Verification

因此两者不是替代关系。Ralph 提供“持续工作”的外循环；Anthropic 2026 Harness 进一步把 planning、definition of done 和 independent evaluation 变成显式角色和 artifact。成熟系统完全可以 是“Ralph-like loop 中嵌入独立 Evaluator”，或进一步升级为 Graph of Loops。

12. 与 Graph Engineering 的关系：这已经是一个 Cyclic Graph



本文架构本身已经不再是简单 while loop，而是带有角色分离、条件边和反馈边的 cyclic graph。真正继续向 Graph Engineering 演进时，最重要的不是增加更多 Agent，而是把 failure routing、fan-out/fan-in、node-local context、checkpoint 和 typed state 变成一等运行时对象。

12.1 为什么“FAIL -> Generator”还不够

失败类别	正确回路
Implementation Error	回 Generator/Coder 修代码
Plan/Scope Error	回 Planner 重做 scope 或优先级
Contract Error	回 Contract Negotiation 重定义可测试 Done
Environment Error	进入 Runtime Recovery，不应让 Coder 猜
Evaluator Uncertain	重新验证 / 更强 judge / Human Gate

从 Loop 到 Graph 的真正升级

不是“多 Agent”，而是“错误应该回到正确节点”。所有失败都 retry Generator，会把计划错误、环境错误和验证错误伪装成代码问题，导致无效循环。

13. 对 OpenCode / 自研长运行 Coding Agent 的可迁移设计

如果把文章思想迁移到 OpenCode 或公司内部 Agent Runtime，最值得复制的是 Harness semantics，而不是 Anthropic 的具体模型、Agent SDK 或前端技术栈。下面给出一个更通用的状态机。



13.1 推荐 P0/P1 能力

优先级	能力	原因
P0	Structured Goal / Product Spec	避免一条高层 prompt 直接进入无限编码。

优先级	能力	原因
P0	Acceptance Contract	编码前固定 Done，阻止 moving goalposts。
P0	Independent Evaluator	把生成与批准权限分离。
P0	Evidence-based PASS	PASS 必须附确定性测试或运行时 evidence。
P0	Failure Classification	失败回到正确节点，避免无意义 retry。
P1	Checkpoint / Resume	长任务失败不从零开始；支持崩溃恢复。
P1	Adaptive Context Policy	根据模型和任务在 Continue / Compact / Reset 之间动态选择。
P1	Eval & Trace Infrastructure	能够复盘 false pass、scope drift、retry 效率。

13.2 不建议一开始就做的东西

- 大量人格化 Agent persona，而没有独立验收机制。
- 复杂 Supervisor Agent，但所有节点仍共享同一巨大上下文。
- 把所有失败都交给“再想一次/再试一次”。
- 只看 LLM 自述完成，不收集测试、运行时和持久化状态证据。
- 把 Fresh Context、Compaction 或 Reset 固化成单一永恒策略。

14. Context Policy：从教条式 Fresh Context 到自适应生命周期

文章最容易被低估的一点是：随着模型从 Sonnet 4.5 到 Opus 4.5，再到 Opus 4.6，原本必须存在的 Context Reset 和 Sprint 被逐步移除。这意味着 Context 管理应该是 policy，而不是固定架构。

状态	建议动作	判断信号
上下文健康	CONTINUE	目标稳定、关键约束可准确回忆、无大量重复探索
轻度污染	COMPACT	旧日志占比高，但当前 reasoning 仍稳定
明显目标漂移/上下文焦虑	RESET + structured handoff	过早收尾、重复忘约束、持续误读旧状态
任务进入全新阶段	RESET 或 node-local context	新阶段只需要旧阶段 artifact，不需要完整历史

```
if context_health == GOOD:
    continue_session()
elif context_health == DEGRADED:
    compact()
elif context_health == POLLUTED or goal_drift:
    persist_structured_state()
    reset_context()
```

核心原则

不要优化“让模型记住一切”，要优化“让模型在当前节点只看到完成当前工作真正需要的状态”。

15. 对文章的批判性审视：仍然存在的 7 个问题

- 15. 证据规模有限：主要是少数复杂应用案例，不是大规模统计 benchmark。
- 16. 成本高：Full Harness 可比 Solo 贵一个数量级以上，只有高价值复杂任务才合理。
- 17. Evaluator 仍是 LLM：它不是 Oracle，仍会漏掉深层 bug、主观质量和环境盲区。
- 18. Planner 是语义单点故障：错误 spec 可以被 Generator 完美实现、Evaluator 完美验收，最终得到“正确执行的错误产品”。
- 19. Rubric 会塑造输出：评分标准可能造成 reward hacking、视觉收敛或 Goodhart 效应。
- 20. 工具决定可见世界：如果 Playwright/视觉/音频/环境接口看不到某类问题，Evaluator 就难以验证它。
- 21. Harness 与模型强耦合：某个模型上有效的复杂度，换模型后可能变成纯开销，因此必须持续重做消融。

15.1 如何补强这些局限

风险	建议补强
Planner 误解	Spec reviewer + requirement traceability + 高风险 human gate
Evaluator false pass	deterministic + behavioral + LLM + hidden tests 多重 grader
Goodhart	部分隐藏验收、随机探索、多个 evaluator、周期性人工校准
成本失控	难度感知的 Harness routing；简单任务走 solo，复杂任务才升级
工具盲区	让 Eval environment 暴露真正 outcome，例如 DB、日志、浏览器、音频/视觉专用指标

16. 如果要把 Harness 做成生产系统，应该测什么

长运行 Harness 不能只测“最终是否完成”。它本身是一套闭环控制系统，因此需要同时测质量、效率、验证可靠性和恢复能力。

指标组	关键指标
Outcome Quality	acceptance pass rate、hidden test pass rate、human acceptance、regression rate
Evaluator Quality	false pass rate、false reject rate、issue recall、judge calibration drift
Loop Efficiency	iterations to pass、重复修改率、无效 retry 比例、failure routing accuracy
Context Health	reset/compaction 次数、重复搜索、约束遗忘、handoff 缺失字段
Cost/Latency	token cost、wall-clock、tool latency、parallel efficiency
Reliability	crash recovery success、checkpoint reuse、resume success、stuck-loop rate

最重要的指标

如果只能先选一个 Harness 质量指标，优先看 False PASS Rate。因为 False PASS 会把错误写入长期状态，并让后续 Agent 把错误当成已经验收的事实。

17. 什么时候应该用 Solo、Loop、Harness、Graph ？

任务类型	推荐结构	判断依据
小修改 / typo / 明确局部 bug	Solo Agent	范围小、验收确定、单次上下文足够。
较长但单路径任务	Verified Loop / Ralph-like	需要多轮修正，但依赖结构简单。
完整应用 / 多小时 build	Planner + Builder + Independent QA	scope 大、容易 stub、需要行为验收。
大型迁移 / 多模块并行 / 多故障域	Graph of Loops	需要 fan-out/fan-in、failure routing、节点级 context 和 checkpoint。

选择原则不是“越高级越好”。最好的 Harness 是当前任务能达到目标所需的最简单结构。复杂度必须由可观察的 failure mode 驱动，而不是由框架功能列表驱动。

18. 文章之后的方向：Dynamic Workflows 进一步验证了“动态 Harness”路线

在这篇 2026 年 3 月文章之后，Anthropic 于 2026 年 5 月底/6 月初推出 Claude Code Dynamic Workflows：Claude 可以针对当前任务动态编写 orchestration script，运行多个并行 subagent，并组合 fan-out/synthesize、adversarial verification、generate/filter、tournament、loop-until-done 等模式。[A6]

这与本文最后的判断高度一致：模型越强，不是所有 Harness 组合都消失；相反，Harness 从“固定补丁”逐渐变成“由任务动态生成的计算拓扑”。这也把 Harness Engineering 自然推向 Graph Engineering。

```
Static Harness (2025)
  Initializer -> Coding Loop

Role-separated Harness (2026-03)
  Planner -> Generator <-> Evaluator

Dynamic Workflow (2026-05/06)
  Task -> generated orchestration graph
        -> parallel workers / critics / filters / loops
        -> evidence -> final result
```

19. 最终结论：这篇文章真正改变了什么

结论 1：Harness 是“错误结构化”

好的 Harness 不是让模型永远不犯错，而是把模型典型错误变成可观察、可分类、可反馈、可恢复的系统状态。

结论 2：Definition of Done 是一等对象

Contract 把“完成”的含义从 Generator 的即时判断中抽离出来，使完成标准可冻结、可测试、可审计。

结论 3：Evaluator 决定闭环上限

Generator 再强，如果 Evaluator 经常 False PASS，长运行系统仍会把错误不断固化进状态。验证能力必须和生成能力同步升级。

结论 4：模型升级必须触发 Harness 消融

模型能力变化会让旧 scaffolding 失去 load-bearing 作用。保持固定复杂架构往往意味着浪费 token、延迟和维护成本。

结论 5：长运行 Coding Agent 的终局更像 Graph of Loops

Planner、Builder、QA、Recovery、Human Gate、parallel workers 各自拥有局部上下文和明确状态接口；Loop 负责局部收敛，Graph 负责全局协调。

如果把全文压缩成一个工程公式，可以写成：

```
Reliable Long-Running Agent
= Model Capability
  x Harness Quality
  x Verification Quality
  x State / Context Discipline
```

因此，提升模型只是四个乘数中的一个。真正的系统工作，是让错误不能轻易穿过验收门，让状态能够跨时间恢复，让控制流在失败时回到正确节点，并持续删除已经不再必要的 scaffolding。

20. 落地检查表：审查一个长运行 Coding Harness 时间这 20 个问题

01. Goal 是否显式、结构化，而不是只存在最初 prompt?
02. 是否有独立 Product Spec/Task Spec?
03. Definition of Done 是否在编码前冻结?
04. Generator 是否能单方面降低验收标准?
05. Evaluator 是否与 Generator 具有独立上下文/角色?
06. PASS 是否要求环境 evidence，而不是 Agent 自述?
07. 是否覆盖真实 E2E 用户路径?
08. 是否同时看 API、数据库、运行时状态，而不只看 UI?
09. 是否记录 Evaluator 的 false pass / false reject?
10. 失败是否分类，还是全部 retry coder?
11. Planning failure 能否回 Planner?
12. Environment failure 是否有专门 recovery?
13. 是否支持 checkpoint / resume?

14. 长期状态是否存在 artifacts/Git，而非只存在 conversation？
15. Context Policy 是否支持 Continue / Compact / Reset？
16. 每个 Agent 是否只获取当前节点所需 context？
17. 是否存在 iteration / time / token budget？
18. 是否有 stuck-loop 和 NEEDS_HUMAN 终态？
19. 模型升级后是否做 Harness ablation？
20. Harness 的质量收益是否真的超过 token、延迟和维护成本？

参考资料与来源

[A1] Anthropic Engineering. “Harness design for long-running application development.” Published 2026-03-24.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>

[A2] Anthropic Engineering. “Effective harnesses for long-running agents.” Published 2025-11-26.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

[A3] Anthropic Engineering. “Effective context engineering for AI agents.” Published 2025-09-29.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

[A4] Anthropic Engineering. “Demystifying evals for AI agents.” Published 2026-01-09.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

[A5] Anthropic. “Introducing Claude Opus 4.6.” Published 2026-02-05. <https://www.anthropic.com/news/claude-opus-4-6>

[A6] Claude by Anthropic. “A harness for every task: dynamic workflows in Claude Code” / “Introducing dynamic workflows in Claude Code.” Published 2026-05-28 / 2026-06-02. <https://claude.com/blog/introducing-dynamic-workflows-in-claude-code>

报告方法说明

- 事实层：优先依据 Anthropic/Claude 官方文章中的架构、数据和作者观察。
- 推演层：控制论、分类器、Goodhart、failure routing、Graph of Loops、OpenCode 迁移等部分属于基于官方材料的工程解释与扩展，报告中已尽量用措辞区分。
- 版权说明：报告对原文采用摘要和转述，不复制文章长段落；示例 contract/状态机/图示均为重新组织的解释性材料。